OpenParEM Installation Manual

Version 2.1

October 2026

Brian Young



Copyright © 2026 Brian Young. All Rights Reserved.

Contents

1 Introduction **2**

2 Validated Operating Systems **2**

3 Installing Pre-compiled Binaries **3**

4 Compiling from Source **3**
4.1 Downloaded Packages . . . 3
4.2 Ubuntu 22 . . . 4
4.3 Using a Downloaded Package . . . 4
4.4 Cloning from GitHub . . . 4

5 Uninstalling **5**

6 OpenParEMg **5**

7 Flow Tools **5**
7.1 FreeCAD . . . 5
7.2 gmsh . . . 6
7.3 ParaView . . . 6

8 Helper Tools and Scripts **6**
8.1 change_slot_count.sh . . . 7
8.2 proj_search . . . 7
8.3 builder . . . 7
8.4 checkError . . . 7
8.5 waveguide . . . 7
8.6 simplify2D . . . 7
8.7 simplify3D . . . 8
8.8 process2D and process2D.sh . . . 8
8.9 process3D and process3D.sh . . . 8
8.10 regression . . . 8
8.11 OpenParEM2D_save.py . . . 8
8.12 OpenParEM3D_save.py . . . 8
8.13 field_plot.py . . . 8
8.14 tline3 . . . 8

9 Execution **9**

10 Specifying Core Counts **9**

11 Computer Considerations for MPI **10**

1 Introduction

OpenParEM consists of the 3D simulator OpenParEM3D, its graphical user interface (GUI) front-end OpenParEMg, and the 2D simulator OpenParEM2D plus some helper tools. All can be compiled from source or installed as pre-compiled bindaries for x86 architectures.

OpenParEMg provides a front-end for building 3D models, setting up 3D simulations, and executing simulations using OpenParEM3D and OpenParEM2D in the background. OpenParEMg does not support plotting or graphing, so external tools are required for these tasks. For plotting S-parameters, any of the free S-parameter plotting tools can be used, or charts can be built in spreadsheet tools. For plots of electromagnetic fields, output is provided for ParaView [4]. ParaView, spreadsheets, or other post-processing tools must be installed separately.

OpenParEMg is not required to run OpenParEM3D. The necessary files to run OpenParEM3D (and OpenParEM2D) can be built from other tools, such as with FreeCAD [1] for 3D modeling, gmsh [2][3] for meshing, scripts for modes and ports, and hand-edited project control files. Examples of projects built using open-source tools other than OpenParEMg are provided in the tutorials and regression test cases.

2 Validated Operating Systems

OpenParEM is validated on a number of Linux flavors for the pre-compiled binary and for compiling from source. Validation in this context means that seven rigorous tests have been passed, and these tests are

- OpenParEM2D regression suite
- OpenParEM3D regression suite
- OpenParEMg regression suite. This is the OpenParEM3D regression suite run from OpenParEMg. Since the meshes between the OpenParEM3D regression suite setups are not identical to the meshes in the OpenParEMg setups, the results vary slightly. For cases with larger differences, the OpenParEMg setup needs maintenance.
- OpenParEMg stress test. A large number of drawing operations are scripted and repeated 1000 times to help ensure that memory issues do not build during long sessions leading to crashes.
- WR90 straight waveguide stress test. The straight waveguide case is build via script and executed. This is repeated 500 times to help ensure that memory isses do not build during long sessions leading to crashes.
- builder regression suite. For the builder tool.
- hand-drawn complex test case

The validation results are summarized in Table 1, showing the results for each for the pre-compiled binary and for compiling from source.

Table 1: Validated Linux Flavors for OpenParEM.

Linux	Version	Pre-compiled Binary	Compiled From Source	Note
Ubuntu	22.04.5 LTS	pass	pass	Source for the pre-compiled binary. See Note 1.
Ubuntu	24.04.5 LTS	pass	pass	
Ubuntu	26.04.1 LTS	pass	pass	
Debian	13 (trixie)	pass	pass	
Fedora	44 (Workstation Edition)	pass	FAIL	See Note 2.
openSUSE	16.0 (Leap)	pass	pass	
Windows WSL2	Ubuntu 26.04.1 LTS	pass	pass	See Notes 3 and 4.

Notes:
1. Compiling for Ubuntu 22 requires additional setup for Python as documented in Sec. 4.2.

2. For Fedora, compiling from source succeeds and passes most tests. However, the antenna regression tests with thin plates immediately crash OpenParEMg on opening the project. Compiled under Fedora, OCCT does not retain sufficient precision to resolve thin plates. Compiler settings would resolve this issue, but to date, the required settings have not been found after multiple attempts.
3. For Windows WSL2, when using the pre-compiled binary, the library libgfortran is required. For WSL2 running Ubuntu, the following can be used to install the needed library.

```
$ sudo apt update
$ sudo apt install gfortran
```

4. For Windows WSL2, the window location cannot be set in the .OpenParEMg file.

3 Installing Pre-compiled Binaries

Pre-compiled binaries are available for x86 Linux and do not require administrator priviledges to install. See Table 1 for Linux flavors for which the pre-compiled binaries have been validated. The basic steps for installing binaries are listed below. See the INSTALL file for more details and installation options.

- Download the installation package OpenParEM-*version*-bin.tar.gz
- Unpack the package:

  ```
  $ tar -xf OpenParEM-version-bin.tar.gz
  $ ln -s OpenParEM-version OpenParEM
  ```

- Change into the base directory and view the installation instructions:

  ```
  $ cd OpenParEM
  $ more INSTALL
  ```

- Set the PATH, FONTCONFIG_PATH, OPAL_PREFIX, and PMIX_PREFIX environment variables. See the INSTALL file for details.
- Install Python scripts for FreeCAD and ParaView:

  ```
  $ cd scripts
  $ OpenParEM\_build --without-all
  ```

  The `--without-all` option installs the scripts while skipping all compilation.
- Test:

  ```
  $ OpenParEM_test
  ```

- Installation is complete if both tests report `pass`.

The pre-compiled binaries are compiled with OpenMPI.

4 Compiling from Source

OpenParEM can be compiled for Linux-based operating systems and does not require administrator priviledges if the needed build tools installed such as g++ and make are installed on the system. See Table 1 for Linux flavors for which compiling has been validated. The OpenParEM_build script checks for the needed building tools and reports any missing tools. After installing any missing tools, the build script can be re-run, and if all tools are found, then compilation proceeds. Note that when installing libtool, two packages are required to be installed: libtool and libtool-bin.

When compiling, OpenMPI is the default, but MPICH can be used instead with a command-line switch. Note that OpenParEM development, testing, and validation is done with OpenMPI. Should issues occur when using MPICH, please contact the developers.

4.1 Downloaded Packages

OpenParEM requires many packages, such as OCCT and Qt, to compile. These are downloaded and checked by the build script OpenParEM_build. Once downloaded, they are not re-downloaded on additional compilation runs, but they are re-checked.

If a download error occurs, then the check will fail, and the build process will stop. On subsequent runs of OpenParEM_build, the process will stop at the faulty download. To clear the faulty download, note the *.gz file on which OpenParEM_build stops, then switch to the `external_packages` directory and delete the gz file and any signature file (sig or asc) file with the matching file name. Then re-run the build script. Barring any download errors, OpenParEM_build should always make it through the download step.

4.2 Ubuntu 22

The library `xkeyboard-config` requires Python 3.11, but due to its age, Ubuntu 22 uses Python 3.10. To compile OpenParEM in Ubuntu 22 requires using Python 3.11 for compiling `xkeyboard-confg`. Python 3.11 must be installed on the system, but it must not be installed for general system use to keep from breaking other applications. The OpenParEM_build script uses the installed Python 3.11 version in a virtual environment to avoid installing it for general use.

To install Python 3.11, issue the following commands (requires root priviledge):

```
$ sudo apt install software-properties-common
$ sudo add-apt-repository ppa:deadsnakes/ppa
$ sudo apt update
$ sudo apt install python3.11 python3.11-dev python3.11-venv
```

During execution, `OpenParEM_build` checks for the presence of Python 3.10, and if it finds it, it checks for the availability of python3.11. If it finds it, processing continues, otherwise an error is printed. Note that this simple check is not intended to cover any of the many possible corner cases: it is just here to get Ubuntu 22 to compile.

4.3 Using a Downloaded Package

For installing using a downloaded installation package, the basic steps for installation are listed below. See the INSTALL file for more details and options.

- Download the installation package OpenParEM-*version*.tar.gz
- Unpack the package:

  ```
  $ tar -xf OpenParEM-version.tar.gz
  $ ln -s OpenParEM-version OpenParEM
  ```

- Change into the base directory and view the installation instructions:

  ```
  $ cd OpenParEM
  $ more INSTALL
  ```

- Change into the scripts directory and compile:

  ```
  $ cd scripts
  $ ./OpenParEM_build --jobs=4
  ```

- Install any build tools that are reported as missing, such as make, g++, etc., then re-run.
- On successful compilation, OpenParEM2D and OpenParEM3D are tested. If both tests report `pass`, then compilation is complete. [Scroll back up the output to see the `pass/FAIL` output.]
- Set the PATH environment variable. See the INSTALL file for details.
- Verify that PATH has been properly set by re-running the tests:

  ```
  $ OpenParEM_test
  ```

- If the two tests report `pass`, then the installation is verified.

4.4 Cloning from GitHub

For installing using using a cloned package from GitHub, the basic steps for installation are listed below. See the INSTALL file for more details and options.

- Clone the project from GitHub. This step requires that git is installed on the system.

```
$ git clone https://github.com/OpenParEM/OpenParEM
```

- Change into the base directory and view the installation instructions:

```
$ cd OpenParEM
$ more INSTALL
```

- Change into the scripts directory and compile:

```
$ cd scripts
$ ./OpenParEM_build --jobs=4
```

- Install any build tools that are reported as missing, such as make, g++, etc., then re-run.
- On successful compilation, OpenParEM2D and OpenParEM3D are tested. If both tests report `pass`, then compilation is complete. [Scroll back up the output to see the `pass/FAIL` output.]
- Set the PATH environment variable. See the INSTALL file for details.
- Verify that PATH has been properly set by re-running the tests:

```
$ OpenParEM_test
```

- If the two tests report `pass`, then the installation is verified.

5 Uninstalling

See the INSTALL file for uninstallation instructions.

6 OpenParEMg

OpenParEMg is a front-end GUI for OpenParEM3D, supporting 3D geometry creation, meshing, simulation setup, and simulation execution. It does not support graphing or plotting since very capable options for these functions already exist as free open-source software. User's will typically install ParaView (see below) for field plots and either a spreadsheet or S-parameter plotting tool for making S-parameter plots.

7 Flow Tools

OpenParEM3D and OpenParEM2D are command-line tools for running electromagnetic simulations only. Pre- and post-processing must be handled by other tools. For pre-processing, OpenParEMg provides a complete front-end. If not using OpenParEMg, then FreeCAD and gmsh along with handy scripts can implement a usable front-end, and their use is described in the tutorials. Many other tools exist that can be used to build a front-end process. For post-processing, ParaView is supported for plotting fields.

7.1 FreeCAD

The geometry can be built with any number of different CAD tools, with FreeCAD [1] being a free and open-source option that works well. FreeCAD is very powerful, but it does have a significant learning curve that pays off as it is mastered. FreeCAD is scriptable with Python, enabling the development of helper scripts to simplify the setup of OpenParEM projects.

The easiest installation process is to install FreeCAD from the software management tool provided by the Linux distribution. Open the tool and search for FreeCAD, and if found, then install. If an option exists, select the FreeCAD version "An open source parametric 3D CAD modeler". However, functionality issues with macros may occur, in which case installing from packages is required.

Alternatively, to install FreeCAD from packages, execute

```
$ sudo add-apt-repository --enable-source \
  ppa:freecad-maintainers/freecad-stable && \
  sudo apt-get update
$ sudo apt-get build-dep freecad
$ sudo apt-get install freecad
```

Installing from packages is fairly slow and also requires administrator priviledges. For Linux flavors not using apt, then substitute the necessary package manager.

The `OpenParEM_build` installation script installs Python scripts to greatly simplify the use of FreeCAD in building OpenParEM projects. The installed location is `$HOME/.FreeCAD/Macro`. Depending on the version of FreeCAD being used, it may be necessary to set the location for the python scripts using the "User macros location:" box in the "Execute macro window" within FreeCAD.

To run FreeCAD, execute

```
$ freecad &
```

On the first use of FreeCAD, set `View`→`Panels`→`Report View` to see messages and errors from the OpenParEM macros during their use.

7.2 gmsh

Since OpenParEM uses the finite element method, a mesh of the geometry to be solved is required. An effective free and open-source mesher is gmsh. Output from FreeCAD can be imported into gmsh and meshed, then the mesh can be saved for use by OpenParEM.

To install gmsh, download the latest version of gmsh from `https://gmsh.info/)` to a suitable installation directory *gmsh-install-dir*, then execute

```
$ cd gmsh-install-dir
$ gunzip gmsh-4.13.1-Linux64.tgz
$ tar -xf gmsh-4.13.1-Linux64.tar
$ cd $HOME/bin
$ ln -s gmsh-install-dir/gmsh-4.13.1-Linux64/bin/gmsh gmsh
```

using the actual downloaded version. If $HOME/bin does not exist, then create it and add it to the PATH environment variable.

To run gmsh, execute

```
$ gmsh -format msh22 &
```

Note that OpenParEM has only been verified with the msh22 format of gmsh due to library limitations of older versions of MFEM.

7.3 ParaView

Both OpenParEM2D and OpenParEM3D can optionally produce outputs to enable viewing of 3D vector fields using ParaView [4], for which there is a free version. It is highly recommended to review the field plots at least initially on any new project. While OpenParEM can use arbitrarily high order finite elements, ParaView is currently limited to finite element orders of 6 and lower.

To install ParaView, execute

```
$ sudo apt-get install paraview
$ sudo apt-get install paraview-doc
$ sudo apt-get install python3-paraview
```

This installation process requires administrator priviledges.

The `OpenParEM_build` installation script installs a Python scripts to greatly simplify the viewing of fields. The installed location is `$HOME/.config/ParaView/Macros`.

To run ParaView, execute

```
$ paraview &
```

8 Helper Tools and Scripts

A number of tools and scripts have been developed to help with setting up projects, project management, code management, and validation.

8.1 change_slot_count.sh

When running a project in OpenParEMg, the number of slots used for execution is given by the keyword/value pair `gui.slot.count N`, where N is the number of slots to use. When running a large number of projects on a new machine that supports a different N, change_slot_count.sh enables N to be changed recursively. For example, to change all of the projects' slot count for use with OpenParEMg from 12 to 6, execute

```
$ cd install_path/OpenParEM/regression
$ change_slot_count.sh . 12 6
```

Projects that are changed are listed.

8.2 proj_search

This is a useful script for finding keywords used in setup files. The format is

```
$ proj_search has|hasnot search_term
```

When executed, the script recursively searches all files ending with ".proj" that either has the search term or does not. For example, to list the finite element order used in all projects in the directory tree, execute

```
$ proj_search has fem.order
```

The output can be further processed using grep. To continue the example, to find all projects that use a mesh order of 6, then execute

```
$ proj_search has fem.order | grep 6
```

8.3 builder

Builder automates the construction of some common transmission line and waveguide types to help with impedance and material studies. Outputs for use with OpenParEM2D or OpenParEM3D are selectable. For more information, see the document "builder_User_Manual.pdf".

8.4 checkError

This code-development script scans the source code and checks error message numbers for missing or duplicate numbers. It is separately run in the OpenParEMCommon, OpenParEM2D, and OpenParEM3D src directories. Note that the error number convention is 1XXX for OpenParEMCommon, 2XXX for OpenParEM2D, and 3XXX for OpenParEM3D. There is no convention for the error number orderings, and they may change from revision to revision.

8.5 waveguide

Waveguide is an unpolished program for calculating propagation constants and characteristic impedances for rectangular waveguide, rectangular waveguide with symmetry, partially-filled rectangular waveguide, coaxial cable, and coaxial waveguide with symmetry. The outputs form the basis for the "exact" answers for many of the cases in the OpenParEM2D regression suite. The only documentation for waveguide is the source code itself. The OpenParEM2D makefile compiles waveguide but does not install it to any location, but it can be run from the source directory or moved to a convenient location.

8.6 simplify2D

simplify2D reduces an OpenParEM2D setup file (*.proj file in the regression suites) to the bare minimum by commenting out inputs that use the default value. The goal is to help identify what is unique about a particular setup.

8.7 simplify3D

simplify3D is the 3D counterpart for OpenParEM3D to simplify2D for OpenParEM2D.

8.8 process2D and process2D.sh

Process2D post-processes OpenParEM2D results and compares against test case values to produce pass/fail evaluations. Test cases are stored in the project directory with the name *project_name*_test_cases.csv. The script process2D.sh automates the use of process2D. To run a project called "my_project" set up with the setup file "my_project.proj" against its test cases, the general command format is

```
$ process2D.sh my_project.proj N
```

where N is the number of cores (or processors) to use for parallel processing. The pass/fail evaluations are saved in the file "my_project_test_results.csv".

8.9 process3D and process3D.sh

Process3D and process3D.sh are the 3D counterparts for OpenParEM3D to process2D and process2D.sh for OpenParEM2D.

8.10 regression

regression automates the execution of the regression suite by running process2D.sh or process3D.sh on a list of projects in the file "regression_case_list.txt" in the directories `install-dir/regression/OpenParEM2D` or `install-dir/regression/OpenParEM3D` . See the "README.txt" files in these directories for more information on running regression.

8.11 OpenParEM2D_save.py

OpenParEM2D_save.py is a Python script that runs in FreeCAD to create the required modes/lines setup file. Building this file by hand is possible but error-prone. The script enables the necessary setup information to be included directly in the FreeCAD drawing, then running OpenParEM2D_save.py as a macro produces the needed setup file quickly and accurately. This script is a "must have" that cannot be overlooked when using FreeCAD. See the OpenParEM2D user's manual for specifications on its use in FreeCAD.

8.12 OpenParEM3D_save.py

OpenParEM3D_save.py is a Python script that runs in FreeCAD to create the required ports setup file. Building this file by hand is theoretically possible but is unmanageable even for near-trivial geometries. The script enables the necessary setup information to be included directly in the FreeCAD drawing, then running OpenParEM3D_save.py as a macro produces the needed setup file quickly and accurately. This script is a "must have" that cannot be overlooked when using FreeCAD. See the OpenParEM3D user's manual for specifications on its use in FreeCAD, which similar but not identical that for OpenParEM2D_save.py.

8.13 field_plot.py

Field_plot.py is a Python script that runs in ParaView to automate the construction of plots for examining the fields produced from OpenParEM2D simulations. Highly recommended.

8.14 tline3

The program tline3 is a custom solver for the cascade of three transmission lines or waveguides with two shunt capacitors at the transitions, and it is used to check matching 3D setups for accuracy. The documentation is in the source code. For examples of use, see the projects
`install-dir/regression/OpenParEM3D/WR90/straight-steps/small`

and
`install-dir/regression/OpenParEM3D/WR90/straight-steps/small-renorm`.

9 Execution

OpenParEMg is simply started at the command line as

```
$ OpenParEMg
```

Some information is scrolled to the starting terminal, particularly from gmsh, and this can be captured into a log file, if desired, with

```
$ OpenParEMg >& OpenParEMg.log
```

Execution of OpenParEM3D and OpenParEM2D are completely managed by OpenParEMg. Even when a project is set up in OpenParEMg, it is always possible to run OpenParEM3D manually as described next.

OpenParEM2D and OpenParEM3D are command-line tools executed with the following commands when parallel processing

```
$ mpirun -q --oversubscribe -np N OpenParEM2D my_project.proj
$ mpirun -q --oversubscribe -np N OpenParEM3D my_project.proj
```

and

```
$ OpenParEM2D my_project.proj
$ OpenParEM3D my_project.proj
```

when running in serial on a single core. Running on a single core is sometimes convenient when getting a new project up-and-running.

When parallel processing, the `-np` option specifies the number of cores to use for the simulation, given by `N`. The `-q` option suppresses messages from the MPI infrastructure, which are rarely needed. The `--oversubscribe` is required when using OpenMPI and when `N` is more than half the number of available cores.

10 Specifying Core Counts

OpenParEM3D and OpenParEM2D can be run using parallel processing to reduce run times, and the user control for this is the number of cores specified for the simulation. The number of cores is set in OpenParEMg in the `Simulation Setup/Options ...` window, and when running at the command line, by the `-np` command line option to `mpirun`.

The number of cores to specify requires some consideration. For simulations on a single computer, `N` should always be set to equal to or less than the number of physical cores. For high-performance computing, there is not sufficient left-over capacity for threads to be utilized. Depending on the computer, setting `N` to the number of physical cores can (and probably will) run slower than if `N` is set to a lower number. The reason is that the available bandwidth on the CPU itself and to/from memory becomes saturated, so adding more cores causes MPI messages to be delayed due to limited bandwidth, slowing overall performance. Experimenting with the value of `N` can identify the value that produces the fastest run times for a given computer. Different computers have different optimal values of `N`. See Sec. 11 for additional discussion.

When running OpenParEM2D with `N` cores, `N` copies of OpenParEM2D are run in parallel with one instance per core. When running OpenParEM3D with `N` cores, `N` copies of OpenParEM3D are run with one instance per core, *plus* `N` copies of OpenParEM2D are run with one instance per core while ports are being solved. For some times during the simulation, 2*`N` cores are being utilized. If running MPI with OpenMPI and if 2`N` > the number of physical cores, then OpenMPI issues an error and exits. Specifying `--oversubscribe` allows processing to continue without errors. OpenParEM3D sleeps while OpenParEM2D runs, so the actual load on the computer is only slightly more than `N`.

11 Computer Considerations for MPI

Ultimately, bandwidth saturation limits the performance gains available from increasing the number of cores applied to a simulation. At some number of cores, the performance gains level out then begin to fall as the number of cores increase further. Maximizing bandwidth is the key to increasing parallel performance with MPI.

For an existing computer, the primary upgrade to improving MPI parallel performance is to fully populate all memory channels with memory sticks. The memory controller will take advantage of the additional memory channels, improving memory bandwidth and reducing run times. This works even if the additional memory capacity is not actually needed. For example, if a computer has two memory channels but only one memory stick, then 1/2 of the memory bandwidth is not being utilized, and adding a second memory stick on the other memory channel can gain significant performance benefits.

If specifying a computer for high-performance computing with MPI, target computers with the largest number of memory channels. For desktops, the number may max out at 4, while for workstations, the number of channels tend to max out at 8. A dual-processor workstation will double the number of memory channels since each processor has its own set, and MPI is smart enough to split the processing across the processors to utilize the extra memory channels. As of this writing, a high-end desktop PC will have a single processor with 4 memory channels, while a high-end workstation will have two processors with 16 memory channels. For parallel processing with MPI, such a workstation handily beats the high-end PC.

Maximum clock rate does play a role, and higher clock speeds do lead to faster run times for a given memory bandwidth. High-end desktop PCs tend to have higher clock frequencies than workstations, and for simulations using low numbers of cores, the high-end PC may produce faster run times than a high-end workstation. However, the PC saturates its bandwidth more quickly with increasing core count than the workstation due to memory bandwidth saturation, and at some number of cores, the workstation begins to produce faster run times, then it can continue driving run times down with increasing core counts until its memory bandwidth saturates. For a workstation, more memory channels trumps higher clock rates.

To recap, populate all memory channels with memory sticks to maximize bandwidth. If specifying a computer, prioritize memory channel counts over clock rates. If budget allows, go for maximum memory channels *and* maximum clock rate.

References

[1] `https://www.freecad.org` version 0.21

[2] C. Geuzaine and J.-F. Remacle, "Gmsh: a three-dimensional finite element mesh generator with built-in pre- and post-processing facilities," *International Journal for Numerical Methods in Engineering*, 79(11), pp. 1309-1331, 2009.

[3] `https://gmsh.info` version 4.13

[4] `https://www.paraview.org`

[5] `https://github.com`